HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

❖

MÔN HỌC: PHÁT TRIỂN HỆ THỐNG THÔNG MINH
(INTELLIGENT SYSTEM DEVELOPMENT)

BÁO CÁO BÀI TẬP LỚN SỐ 05

CÁC KIẾN TRÚC MẠNG NƠ-RON TÍCH CHẬP (CNN) TIÊU BIỂU VÀ ĐÁNH GIÁ THỰC NGHIỆM ĐA TẬP DỮ LIỆU

Khảo sát và đối chuẩn 4 mô hình (Basic CNN, LeNet, VGG-style, ResNet-style) trên MNIST, Fashion-MNIST và dữ liệu bảng BRFSS 2015 Diabetes (Conv1D) bằng PyTorch

Sinh viên thực hiện:	**Nguyễn Thành Trung**
Mã số sinh viên:	B23DCCN861
Lớp chuyên ngành:	D23CTPM01-B
Giảng viên hướng dẫn:	PGS. TS. Trần Đình Quế (Dinh Que Tran, Ph.D.)

Hà Nội, Tháng 09 năm 2026

MỤC LỤC

1. GIỚI THIỆU ASSIGNMENT 05 4
1.1. Bối cảnh và mục tiêu nghiên cứu 4
1.2. Nhiệm vụ và phạm vi thực nghiệm 4
2. CƠ SỞ LÝ THUYẾT CNN VÀ CÁC KIẾN TRÚC MỞ RỘNG 5
2.1. 14 Khái niệm nền tảng trong Mạng nơ-ron Tích chập 5
2.2. Khảo sát dòng biến đổi kích thước Tensor (Tensor Shapes Walkthrough) 7
2.3. Kiến trúc Basic CNN (Chuẩn hóa cơ bản) 8
2.4. Kiến trúc LeNet-style CNN (Yann LeCun 1998) 9
2.5. Kiến trúc VGG-style CNN (Simonyan & Zisserman 2014) 10
2.6. Kiến trúc ResNet-style CNN (He et al. 2016) 11
3. THIẾT KẾ THỰC NGHIỆM VÀ QUY TRÌNH KIỂM CHỨNG 12
3.1. Môi trường tính toán và cấu hình hạt giống ngẫu nhiên 12
3.2. Quy trình tiền xử lý, phân chia dữ liệu và chống rò rỉ thông tin 13
3.3. Hiện thực chu trình huấn luyện, hàm mất mát và Early Stopping 14
3.4. Phương pháp suy luận và hệ thống các chỉ số đánh giá đa chiều 15
4. THỰC NGHIỆM 1 — TẬP DỮ LIỆU MNIST (CHỮ SỐ VIẾT TAY) 17
4.1. Đặc trưng dữ liệu và trực quan hóa mẫu chữ số 17
4.2. Quá trình huấn luyện và đường cong hội tụ thực tế 17
4.3. Bảng số liệu đối chuẩn và phân tích ma trận nhầm lẫn 18
5. THỰC NGHIỆM 2 — TẬP DỮ LIỆU FASHION-MNIST (SẢN PHẨM THỜI TRANG) 19
5.1. Độ phức tạp hình học và trực quan hóa mẫu sản phẩm 19
5.2. Quá trình huấn luyện và hiện tượng phân tách đặc trưng 20
5.3. Bảng số liệu đối chuẩn và phân tích nhầm lẫn biên cạnh 20
6. THỰC NGHIỆM 3 — TẬP DỮ LIỆU DIABETES (DỮ LIỆU BẢNG VỚI CONV1D) 23
6.1. Đặc trưng lâm sàng, phân bố mất cân bằng và bản chất sư phạm 23
6.2. Thiết kế các mô hình Conv1D và tiến trình huấn luyện 24
6.3. Bẫy số đo Accuracy và đánh giá thực chất qua Macro-F1 / ROC-AUC 25
7. SO SÁNH ĐỐI CHUẨN 4 KIẾN TRÚC TRÊN 3 MIỀN DỮ LIỆU 27
7.1. Bảng ma trận tổng hợp 12 mô hình thực nghiệm 27
7.2. Sự dịch chuyển hiệu năng giữa miền Thị giác máy tính và Dữ liệu bảng 28
8. PHÂN TÍCH CHUYÊN SÂU CÁC YẾU TỐ KỸ THUẬT 29
8.1. Tương quan giữa số lượng tham số và năng lực biểu diễn 29
8.2. Phân tích nghịch lý thời gian huấn luyện trên CPU 29
8.3. Độ nhạy của các thước đo trong điều kiện mất cân bằng nghiêm trọng 30
9. THẢO LUẬN TỔNG HỢP (CROSS-DATASET SYNTHESIS) 30
9.1. Cơ chế trích xuất đặc trưng không gian 2D của CNN 30
9.2. Vì sao CNN tự nhiên tối ưu hóa trên ảnh nhưng bị giới hạn trên bảng? 30
9.3. Phân tích bước nhảy kiến trúc: Từ LeNet qua VGG tới ResNet 31
9.4. Cơ chế toán học của Skip Connection và giải pháp triệt tiêu Vanishing Gradient 31
9.5. Đánh đổi (Trade-off) giữa độ sâu, chi phí tính toán và độ tổng quát hóa 31
10. HẠN CHẾ CỦA BÀI TOÁN VÀ HƯỚNG PHÁT TRIỂN 32
11. KẾT LUẬN 32
12. TÀI LIỆU THAM KHẢO 33

1. GIỚI THIỆU ASSIGNMENT 05

1.1. Bối cảnh và mục tiêu nghiên cứu

Mạng nơ-ron tích chập (Convolutional Neural Networks - CNN) đã tạo nên cuộc cách mạng căn bản trong lĩnh vực Trí tuệ nhân tạo và Thị giác máy tính kể từ khi mô hình LeNet-5 của Yann LeCun (1998) ra đời và bùng nổ mạnh mẽ qua các cột mốc như AlexNet (2012), VGGNet (2014) và ResNet (2015). Khác với các mạng truyền thẳng (Fully Connected Multi-Layer Perceptrons) có xu hướng bị bùng nổ tham số và phá vỡ cấu trúc không gian của dữ liệu, CNN kế thừa các nguyên lý sinh học của vỏ não thị giác (Visual Cortex) với hai cơ chế then chốt: vùng cảm nhận cục bộ (Local Receptive Fields) và chia sẻ trọng số (Weight Sharing). Nhờ đó, CNN sở hữu tính chất bất biến đối với phép dịch chuyển (Translation Equivariance), cho phép trích xuất phân tầng các đặc trưng từ cơ bản (cạnh, góc, kết cấu) đến phức tạp (bộ phận, hình thái toàn thể).

Mục tiêu trọng tâm của Assignment 05 trong học phần Phát triển Hệ thống Thông minh (Intelligent System Development) là:

- Làm chủ 14 khái niệm cơ bản cấu thành nên kiến trúc và chu trình vận hành của CNN, liên kết chặt chẽ giữa lý thuyết toán học và các lớp/hàm lập trình tương ứng trong framework PyTorch hiện đại.
- Thiết kế, hiện thực hóa và phân tích 4 họ kiến trúc CNN có tính biểu tượng lịch sử và ứng dụng thực tiễn cao: Basic CNN (chuẩn hóa baseline), LeNet-style (kiến trúc tiên phong với kernel 5x5 và subsampling), VGG-style (xếp chồng filter nhỏ 3x3 kèm Batch Normalization và Dropout), và ResNet-style (kết nối tắt residual skip connections và Global Average Pooling).
- Mở rộng khảo sát thực nghiệm trên 3 tập dữ liệu đại diện cho các miền bài toán khác nhau: Nhận diện chữ số viết tay (MNIST), Nhận diện sản phẩm thời trang có chi tiết hình học phức tạp (Fashion-MNIST), và Phân loại nguy cơ bệnh tiểu đường (BRFSS 2015 Diabetes) dạng bảng thông qua mô hình Conv1D.
- Thực hiện đo lường định lượng 12 mô hình thực nghiệm độc lập dựa trên bộ chỉ số toàn diện: Accuracy, Precision, Recall, Macro-F1, ROC-AUC, số lượng tham số khả huấn luyện và thời gian thực thi thực tế trên CPU.

1.2. Nhiệm vụ và phạm vi thực nghiệm

Báo cáo này được cấu trúc để phản ánh trung thực toàn bộ kết quả thu được từ quá trình thực thi mã nguồn độc lập. Đặc biệt, đối với tập dữ liệu y tế Diabetes BRFSS 2015, báo cáo duy trì sự phân định minh bạch tuyệt đối giữa tập dữ liệu khảo sát gốc (Raw BRFSS

gồm 253,680 bản ghi) và phân tập mô hình hóa thực nghiệm của Assignment 05 (Modeling Subset gồm 40,000 bản ghi được lấy mẫu phân tầng ngẫu nhiên có đại diện cao, phân tách thành 28,000 mẫu Huấn luyện, 6,000 mẫu Kiểm định và 6,000 mẫu Kiểm thử). Thí nghiệm trên dữ liệu bảng được thiết kế và phân tích dưới góc độ một thí nghiệm sư phạm (Pedagogical Teaching Experiment), giúp sinh viên so sánh sâu sắc sự khác biệt về giả định quy nạp (Inductive Bias) giữa dữ liệu có cấu trúc lưới 2D tự nhiên và dữ liệu dạng bảng có các cột thuộc tính rời rạc.

2. CƠ SỞ LÝ THUYẾT CNN VÀ CÁC KIẾN TRÚC MỞ RỘNG

2.1. 14 Khái niệm nền tảng trong Mạng nơ-ron Tích chập

Mạng nơ-ron tích chập được xây dựng từ sự kết hợp hài hòa của các khối chức năng toán học. Bảng 1 tóm tắt toàn diện 14 khái niệm cơ bản theo yêu cầu chuẩn, đối chiếu trực tiếp với các lớp và hàm tương ứng trong thư viện PyTorch.

Bảng 1. Bảng tra cứu 14 khái niệm cơ bản của CNN và hiện thực tương ứng trong PyTorch.

Khái niệm	Giải thích bản chất kỹ thuật	Lớp/Hàm tương ứng trong PyTorch
1. Convolution	Phép trượt nhân chập giữa ma trận đầu vào và ma trận trọng số (kernel) để tạo bản đồ đặc trưng.	torch.nn.Conv2d / Conv1d
2. Kernel/Filter	Ma trận trọng số khả học có kích thước nhỏ (ví dụ 3x3, 5x5) quét qua dữ liệu để phát hiện đặc trưng cục bộ.	torch.nn.Parameter trong Conv
3. Stride	Bước nhảy di chuyển của bộ lọc theo chiều ngang và dọc qua từng phép tính chập.	Tham số stride trong Conv/Pool
4. Padding	Kỹ thuật đệm thêm các giá trị (thường là 0) quanh viền tensor để kiểm soát kích thước output và bảo toàn biên.	Tham số padding trong Conv
5. Feature Map	Ma trận hoặc tensor đầu ra sau phép tích chập, biểu diễn cường độ kích hoạt của đặc trưng học được.	Output tensor của lớp Conv
6. ReLU	Hàm kích hoạt phi tuyến f(x) = max(0, x), giải quyết triệt để vấn đề bão hòa gradient so với hàm Sigmoid/Tanh.	torch.nn.ReLU / F.relu
7. Pooling	Phép lấy mẫu giảm chiều không gian (Max hoặc Average), tăng tính bất biến dịch chuyển và giảm lượng tính toán.	torch.nn.MaxPool2d / AvgPool2d

Khái niệm	Giải thích bản chất kỹ thuật	Lớp/Hàm tương ứng trong PyTorch
8. Flatten	Phép biến đổi kéo phẳng tensor không gian [B, C, H, W] thành vector đặc trưng [B, C*H*W] trước khi vào Dense.	torch.nn.Flatten
9. Fully Connected	Tầng kết nối dày đặc (Dense/Linear), tổng hợp các đặc trưng cục bộ thành biểu diễn toàn cục để phân loại.	torch.nn.Linear
10. Softmax	Hàm chuẩn hóa vector logits thành phân phối xác suất hợp lệ có tổng bằng 1 qua hàm e^z_i / sum(e^z_j).	torch.nn.Softmax / F.softmax
11. Loss	Hàm đo lường độ lệch giữa phân phối xác suất dự đoán và nhãn thực tế, tiêu biểu là CrossEntropyLoss.	torch.nn.CrossEntropyLoss
12. Forward Prop	Chu trình truyền tín hiệu từ tensor đầu vào qua các tầng tích chập, phi tuyến, gộp đến vector đầu ra.	Phương thức forward(x)
13. Backpropagation	Thuật toán lan truyền ngược đạo hàm sai số từ hàm mất mát về từng trọng số theo quy tắc dây chuyền.	loss.backward()
14. Optimizer	Thuật toán cập nhật trọng số dựa trên gradient tích lũy (ví dụ Adam kết hợp quán tính và thích ứng bước nhảy).	torch.optim.Adam / SGD

Đoạn mã sau đây minh họa trực tiếp cách khai báo và thực thi một chuỗi xử lý cơ bản bao gồm đầy đủ các khối chức năng từ tích chập, kích hoạt phi tuyến ReLU, gộp cực đại MaxPool đến duỗi phẳng Flatten trong PyTorch:

Listing 1. Minh họa luồng tính toán qua các khối CNN cơ bản trong PyTorch.

```
import torch
import torch.nn as nn

# 1. Khởi tạo một mẫu tensor ảnh đầu vào giả định [Batch=2, Channels=1, Height=28,
Width=28]
x = torch.randn(2, 1, 28, 28)

# 2. Khai báo các khối thành phần cốt lõi của CNN
conv = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, stride=1, padding=1)
relu = nn.ReLU()
pool = nn.MaxPool2d(kernel_size=2, stride=2)
flatten = nn.Flatten()
fc = nn.Linear(in_features=16 * 14 * 14, out_features=10)

# 3. Lan truyền tiến và in kích thước tensor qua từng tầng
out_conv = conv(x)       # -> [2, 16, 28, 28]
out_relu = relu(out_conv)# -> [2, 16, 28, 28]
out_pool = pool(out_relu)# -> [2, 16, 14, 14]
out_flat = flatten(out_pool) # -> [2, 3136]
logits = fc(out_flat)    # -> [2, 10]
```

```
print('Kích thước logits cuối cùng:', logits.shape)
```

Giải thích code:

• `nn.Conv2d(1, 16, kernel_size=3, padding=1)`: Tiếp nhận tensor ảnh 1 kênh, áp dụng 16 bộ lọc kích thước 3x3 với đệm padding=1, giữ nguyên kích thước không gian 28x28 và sinh ra 16 bản đồ đặc trưng.
• `nn.ReLU()`: Áp dụng phi tuyến tính từng phần tử max(0, x), giữ nguyên kích thước tensor [2, 16, 28, 28].
• `nn.MaxPool2d(2, 2)`: Lấy mẫu cực đại trong cửa sổ 2x2 với bước nhảy stride=2, giảm độ phân giải xuống còn [2, 16, 14, 14].
• `nn.Flatten()`: Trải phẳng các chiều không gian (16 * 14 * 14 = 3136) thành vector đặc trưng [2, 3136].
• `nn.Linear(3136, 10)`: Tầng kết nối đầy đủ ánh xạ biểu diễn đặc trưng sang 10 giá trị logit phân loại.

2.2. Khảo sát dòng biến đổi kích thước Tensor (Tensor Shapes Walkthrough)

Kích thước đầu ra của một tầng tích chập 2D được xác định chặt chẽ bằng công thức giải tích sau:

H_out = floor((H_in + 2*P - K) / S) + 1
W_out = floor((W_in + 2*P - K) / S) + 1

Trong đó H_in, W_in là chiều cao và độ rộng đầu vào; K là kích thước kernel; P là lượng padding; S là giá trị stride. Bảng 2 trình bày chi tiết sự biến đổi hình học của tensor kích thước [B, 1, 28, 28] qua từng tầng của mô hình Basic CNN.

Bảng 2. Bảng theo dõi dòng kích thước Tensor qua từng giai đoạn của Basic CNN.

Giai đoạn / Tầng xử lý	Kích thước Tensor [Batch, C, H, W]	Ý nghĩa biểu diễn hình học
Đầu vào (Input)	[B, 1, 28, 28]	Ảnh gốc đơn kênh thang độ xám (grayscale)
Conv1 (k=3, p=1, s=1)	[B, 16, 28, 28]	16 bộ lọc 3x3 quét trích xuất đặc trưng bậc thấp, bảo toàn độ phân giải nhờ p=1
ReLU1	[B, 16, 28, 28]	Khử toàn bộ giá trị âm max(0, x), giữ nguyên kích thước không gian
MaxPool1 (k=2, s=2)	[B, 16, 14, 14]	Giảm một nửa chiều cao và chiều rộng, chọn giá trị nổi bật nhất trong ô 2x2
Conv2 (k=3, p=1, s=1)	[B, 32, 14, 14]	32 bộ lọc trích xuất tổ hợp đặc trưng bậc cao hơn

Giai đoạn / Tầng xử lý	Kích thước Tensor [Batch, C, H, W]	Ý nghĩa biểu diễn hình học
ReLU2	[B, 32, 14, 14]	Phi tuyến hóa không gian đặc trưng tầng thứ hai
MaxPool2 (k=2, s=2)	[B, 32, 7, 7]	Nén không gian về kích thước 7x7, tổng cộng 32 bản đồ đặc trưng
Flatten	[B, 1568]	Trải phẳng toàn bộ đặc trưng: 32 * 7 * 7 = 1568 phần tử
Dense Hidden (Linear + ReLU)	[B, 64]	Tầng kết nối ẩn gồm 64 nơ-ron tổng hợp toàn cục
Output Logits (Linear)	[B, 10]	10 giá trị logit chưa chuẩn hóa tương ứng với 10 lớp phân loại

2.3. Kiến trúc Basic CNN (Chuẩn hóa cơ bản)

Kiến trúc Basic CNN đóng vai trò là mô hình đường cơ sở (Baseline). Thiết kế bao gồm 2 giai đoạn tích chập - kích hoạt - gộp (Conv-ReLU-Pool), tiếp nối bởi một đầu phân loại (Classification Head) gồm 2 tầng tuyến tính fully connected. Mô hình không sử dụng chuẩn hóa Batch Normalization hay Dropout, giúp theo dõi độ hội tụ tự nhiên của các tầng tích chập.

Listing 2. Định nghĩa kiến trúc BasicCNN2D trong file src/assignment05/models.py.

```
class BasicCNN2D(nn.Module):
    def __init__(self, in_channels: int = 1, num_classes: int = 10):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, 16, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(32 * 7 * 7, 64)
        self.relu3 = nn.ReLU()
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = self.flatten(x)
        x = self.relu3(self.fc1(x))
        return self.fc2(x)
```

Giải thích code:

• `Stage 1 (conv1, relu1, pool1)`: Biến đổi tensor từ [B, 1, 28, 28] thành [B, 16, 14, 14], thực hiện trích xuất các cạnh cơ bản.
• `Stage 2 (conv2, relu2, pool2)`: Nhân đôi số kênh đặc trưng lên 32 và giảm kích thước không gian còn [B, 32, 7, 7].
• `Head (flatten, fc1, relu3, fc2)`: Vector hóa 1568 chiều, đưa qua tầng ẩn 64 nơ-ron trước khi phân lớp ra 10 logits.

2.4. Kiến trúc LeNet-style CNN (Yann LeCun 1998)

Mô hình LeNet-style mô phỏng sát kiến trúc kinh điển LeNet-5. Điểm khác biệt quan trọng so với Basic CNN gồm:
1. Kích thước Kernel lớn hơn: Sử dụng bộ lọc 5x5 thay vì 3x3, giúp mở rộng trường tiếp nhận (Receptive Field) ngay tại tầng đầu.
2. Lấy mẫu trung bình (Average Pooling): Thay vì MaxPool, LeNet-5 truyền thống sử dụng Subsampling/AvgPool, tính giá trị trung bình trong cửa sổ 2x2, tạo hiệu ứng làm mịn (smoothing) các bản đồ đặc trưng.
3. Đầu phân loại phân tầng sâu: Sử dụng 3 tầng Fully Connected liên tiếp (120 -> 84 -> 10) phản ánh tư duy nén thông tin tuần tự.

Listing 3. Định nghĩa kiến trúc LeNetCNN2D trong file src/assignment05/models.py.

```
class LeNetCNN2D(nn.Module):
    def __init__(self, in_channels: int = 1, num_classes: int = 10):
        super().__init__()
        # Stage 1: Conv 5x5 padding=2 bảo toàn 28x28 -> AvgPool 2x2 -> 14x14
        self.conv1 = nn.Conv2d(in_channels, 6, kernel_size=5, padding=2)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
        # Stage 2: Conv 5x5 padding=0 -> 10x10 -> AvgPool 2x2 -> 5x5
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, padding=0)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.relu3 = nn.ReLU()
        self.fc2 = nn.Linear(120, 84)
        self.relu4 = nn.ReLU()
        self.fc3 = nn.Linear(84, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.pool1(self.relu1(self.conv1(x)))
        x = self.pool2(self.relu2(self.conv2(x)))
        x = self.flatten(x)
        x = self.relu3(self.fc1(x))
        x = self.relu4(self.fc2(x))
        return self.fc3(x)
```

Giải thích code:

• `conv1 (5x5, p=2)`: Kernel 5x5 bao quát diện tích 25 điểm ảnh lân cận, padding=2 giữ kích thước 28x28 trước khi đưa vào AvgPool.
• `conv2 (5x5, p=0)`: Không dùng padding, kích thước từ 14x14 giảm tự nhiên xuống 10x10, sau đó AvgPool đưa về 5x5.
• `Phân tầng FC (120 -> 84 -> 10)`: Cơ chế giảm chiều nơ-ron dần dần giúp nén và cô đọng biểu diễn toàn cục.

2.5. Kiến trúc VGG-style CNN (Simonyan & Zisserman 2014)

Triết lý thiết kế cốt lõi của VGGNet (Visual Geometry Group, Oxford) nằm ở việc thay thế các bộ lọc kích thước lớn (như 5x5 hay 7x7) bằng chuỗi các tầng tích chập nhỏ 3x3 xếp chồng liên tiếp. Về mặt toán học, hai tầng tích chập 3x3 liên tiếp tạo ra một trường tiếp nhận hiệu dụng (Effective Receptive Field) tương đương chính xác một tầng 5x5:

RF = 3 + (3 - 1) = 5

Tuy nhiên, giải pháp xếp chồng 3x3 mang lại hai ưu thế vượt trội:
• Giảm mạnh tham số: Hai tầng 3x3 với C kênh tiêu tốn 2 * (3 * 3 * C^2) = 18 C^2 trọng số, trong khi một tầng 5x5 tiêu tốn 1 * (5 * 5 * C^2) = 25 C^2 trọng số (tiết kiệm 28% tham số).
• Gia tăng phi tuyến: Việc chèn hai hàm kích hoạt ReLU thay vì một giúp mô hình học được các hàm quyết định phức tạp và sắc nét hơn.

Ngoài ra, mô hình VGG-style được trang bị Batch Normalization sau mỗi phép chập để ổn định phân phối dữ liệu nội tại và Dropout (tỷ lệ 0.4) tại tầng Dense nhằm chống hiện tượng học vẹt (overfitting).

Listing 4. Định nghĩa kiến trúc VGGCNN2D trong file src/assignment05/models.py.

```
class VGGCNN2D(nn.Module):
    def __init__(self, in_channels: int = 1, num_classes: int = 10, dropout: float = 0.4):
        super().__init__()
        # Block 1: 2 lớp Conv 3x3 liên tiếp + BatchNorm + MaxPool
        self.block1 = nn.Sequential(
            nn.Conv2d(in_channels, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16), nn.ReLU(),
            nn.Conv2d(16, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        # Block 2: 2 lớp Conv 3x3 liên tiếp + BatchNorm + MaxPool
        self.block2 = nn.Sequential(
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(32 * 7 * 7, 128), nn.ReLU(),
            nn.Dropout(p=dropout),
            nn.Linear(128, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.classifier(self.block2(self.block1(x)))
```

Giải thích code:

• `Sequential Block1 & Block2`: Mỗi khối gồm 2 tầng Conv2d(3x3) liên tiếp kèm BatchNorm2d và ReLU, sau đó mới qua MaxPool2d.

• `nn.BatchNorm2d`: Chuẩn hóa giá trị kích hoạt theo từng mini-batch, triệt tiêu hiện tượng dịch chuyển hiệp biến nội (Internal Covariate Shift).

• `nn.Dropout(0.4)`: Ngẫu nhiên tắt 40% kết nối nơ-ron trong quá trình huấn luyện, ép mạng phải học các biểu diễn phân tán mạnh mẽ.

2.6. Kiến trúc ResNet-style CNN (He et al. 2016)

ResNet (Residual Networks) giải quyết vấn đề suy thoái mạng sâu (Degradation Problem) và nguy cơ tiêu biến đạo hàm (Vanishing Gradient) khi tăng số tầng. Thay vì bắt các tầng ẩn phải học trực tiếp ánh xạ mục tiêu H(x), ResNet cấu trúc lại bài toán thành việc học ánh xạ phần dư (Residual Mapping):

F(x) = H(x) - x ==> y = F(x) + x

Phép toán cộng trực tiếp x (Identity Shortcut / Skip Connection) tạo nên một đường truyền trực tiếp cho luồng đạo hàm lan truyền ngược:

dLoss / dx = (dLoss / dy) * (dF(x) / dx + 1)

Số hạng +1 đóng vai trò như một 'đại lộ thông tin' (Gradient Highway), giúp giảm thiểu đáng kể nguy cơ tiêu biến gradient so với các mạng truyền thẳng thông thường khi đi qua nhiều tầng tính toán. Một cải tiến quan trọng khác của ResNet là việc loại bỏ hoàn toàn tầng Flatten cồng kềnh, thay bằng tầng Gộp trung bình toàn cục (Global Average Pooling - GAP), nén mỗi bản đồ đặc trưng 7x7 thành một giá trị vô hướng duy nhất trước khi đưa vào phân loại.

Listing 5. Định nghĩa khối ResidualBlock2D và mô hình ResNetCNN2D trong src/assignment05/models.py.

```
class ResidualBlock2D(nn.Module):
    def __init__(self, in_channels: int, out_channels: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3, stride=stride, padding=1,
```

```
bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3, stride=1, padding=1,
bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.relu(self.bn2(self.conv2(self.relu(self.bn1(self.conv1(x))))) +
self.shortcut(x))

class ResNetCNN2D(nn.Module):
    def __init__(self, in_channels: int = 1, num_classes: int = 10):
        super().__init__()
        self.initial = nn.Sequential(
            nn.Conv2d(in_channels, 16, 3, padding=1, bias=False),
            nn.BatchNorm2d(16), nn.ReLU()
        )
        self.layer1 = ResidualBlock2D(16, 16, stride=1)
        self.layer2 = ResidualBlock2D(16, 32, stride=2)
        self.layer3 = ResidualBlock2D(32, 64, stride=2)
        self.gap = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(64, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.layer3(self.layer2(self.layer1(self.initial(x))))
        return self.fc(torch.flatten(self.gap(x), 1))
```

Giải thích code:

• `ResidualBlock2D`: Thực hiện phép cộng out + shortcut(x). Nếu số kênh thay đổi hoặc stride=2, nhánh shortcut áp dụng Conv2d(1x1) kèm BatchNorm để đồng bộ kích thước tensor trước khi cộng.

• `bias=False`: Không cần trọng số bias trong lớp Conv2d khi ngay sau đó là lớp BatchNorm2d, giúp tiết kiệm tham số.

• `nn.AdaptiveAvgPool2d((1, 1))`: Thu gọn mọi kích thước không gian [B, 64, 7, 7] về [B, 64, 1, 1], giảm kích thước đầu phân loại chỉ còn 650 tham số.

3. THIẾT KẾ THỰC NGHIỆM VÀ QUY TRÌNH KIỂM CHỨNG

3.1. Môi trường tính toán và cấu hình hạt giống ngẫu nhiên

Thực nghiệm được thiết lập trên môi trường máy tính để bàn sử dụng CPU x86-64, hệ điều hành Microsoft Windows 11, phiên bản Python 3.12.3 và thư viện PyTorch

2.13.0+cpu. Nhằm đảm bảo tính tái lập 100% của toàn bộ kết quả, hạt giống ngẫu nhiên (random seed) được cố định đồng nhất tại giá trị 42 cho toàn bộ các thư viện liên quan bao gồm PyTorch (`torch.manual_seed(42)`), NumPy (`np.random.seed(42)`) và Scikit-Learn.

3.2. Quy trình tiền xử lý, phân chia dữ liệu và chống rò rỉ thông tin

Để đánh giá khách quan năng lực tổng quát hóa của các mô hình, toàn bộ 3 tập dữ liệu được phân tách theo tỷ lệ phân tầng (Stratified Splitting) nhằm giữ nguyên tỷ lệ phân bố giữa các nhãn lớp:

• MNIST & Fashion-MNIST: 70,000 ảnh chuẩn hóa pixel về [0.0, 1.0], phân chia 49,000 Train / 10,500 Val / 10,500 Test.

• Diabetes BRFSS 2015: Tập dữ liệu gốc gồm 253,680 dòng. Để đảm bảo tính khả thi thực nghiệm trên CPU trong phạm vi bài tập, phân tập mô hình hóa Assignment 05 (Modeling Subset) sử dụng 40,000 bản ghi lấy mẫu phân tầng ngẫu nhiên, phân chia thành: Train = 28,000 (70%), Validation = 6,000 (15%), Test = 6,000 (15%).

Listing 6. Quy trình tiền xử lý, phân chia dữ liệu và chuẩn hóa fit train-only trong src/assignment05/data.py.

```
# Trích đoạn từ src/assignment05/data.py: Nạp, phân chia phân tầng và chuẩn hóa
def load_diabetes_data(test_size=0.15, val_size=0.15, max_samples=40000, seed=42):
    df = pd.read_csv(csv_path)
    if max_samples is not None and max_samples < len(df):
        df, _ = train_test_split(
            df, train_size=max_samples, random_state=seed, stratify=df['Diabetes_binary']
        )
    x_raw = df[feature_cols].values.astype(np.float32)
    y_raw = df['Diabetes_binary'].values.astype(np.int64)

    # Phân chia phân tầng: Train (28,000), Val (6,000), Test (6,000)
    x_train_val, x_test, y_train_val, y_test = train_test_split(
        x_raw, y_raw, test_size=0.15, random_state=seed, stratify=y_raw
    )
    x_train, x_val, y_train, y_val = train_test_split(
        x_train_val, y_train_val, test_size=val_size / 0.85, random_state=seed,
stratify=y_train_val
    )

    # BẮT BUỘC: Fit scaler CHỈ trên Train set để chống Data Leakage
    scaler = StandardScaler()
    x_train = scaler.fit_transform(x_train)
    x_val = scaler.transform(x_val)    # Dùng đúng mean/std của Train
    x_test = scaler.transform(x_test)  # Dùng đúng mean/std của Train

    # Định hình lại tensor 3 chiều [N, Channels=1, Length=21] cho Conv1D
    x_train = x_train[:, np.newaxis, :].astype(np.float32)
    x_val = x_val[:, np.newaxis, :].astype(np.float32)
    x_test = x_test[:, np.newaxis, :].astype(np.float32)
    return (x_train, y_train), (x_val, y_val), (x_test, y_test)
```

Giải thích code:

• `train_test_split(..., stratify=y)`: Duy trì nghiêm ngặt tỷ lệ phân bố lớp (86.07% lớp 0 vs 13.93% lớp 1) trên cả 3 tập Train (28,000), Val (6,000) và Test (6,000).
• `scaler.fit_transform(x_train)`: Chỉ tính toán kỳ vọng μ và độ lệch chuẩn σ trên tập huấn luyện Train.
• `scaler.transform(x_val)` & `scaler.transform(x_test)`: Áp dụng cùng tham số chuẩn hóa của Train sang Val và Test. Tuyệt đối không fit lại trên tập Val/Test nhằm triệt tiêu nguy cơ rò rỉ dữ liệu (Data Leakage).
• `x[:, np.newaxis, :]`: Mở rộng chiều tensor từ [N, 21] thành [N, 1, 21] để tương thích với cấu trúc đầu vào của các lớp tích chập Conv1D.

3.3. Hiện thực chu trình huấn luyện, hàm mất mát và Early Stopping

Toàn bộ các mô hình được huấn luyện bằng hàm `train_model` trong `src/assignment05/training.py`. Đoạn mã dưới đây minh họa chu trình tối ưu hóa hoàn chỉnh từ pha huấn luyện, lan truyền ngược, cập nhật trọng số Adam đến kiểm định và lưu checkpoint tốt nhất:

Listing 7. Chu trình huấn luyện (Training Loop) với Adam, CrossEntropy và Early Stopping trong src/assignment05/training.py.

```
# Trích đoạn từ src/assignment05/training.py: Chu trình huấn luyện PyTorch chuẩn hóa
def train_model(model, train_loader, val_loader, epochs=6, lr=0.001,
early_stopping_patience=2):
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=1e-4)
    best_score = float('inf')
    best_weights = copy.deepcopy(model.state_dict())

    for epoch in range(1, epochs + 1):
        # 1. PHA HUẤN LUYỆN (TRAINING PHASE)
        model.train()  # Bật training mode (Dropout, BatchNorm update)
        for x_batch, y_batch in train_loader:
            optimizer.zero_grad()            # Xóa gradient batch trước
            logits = model(x_batch)          # Forward propagation
            loss = criterion(logits, y_batch)# Tính hàm mất mát CrossEntropy
            loss.backward()                  # Backpropagation tính gradient
            optimizer.step()                 # Thuật toán Adam cập nhật trọng số

        # 2. PHA KIỂM ĐỊNH (VALIDATION PHASE)
        model.eval()  # Chuyển sang eval mode (đóng băng BatchNorm/Dropout)
        val_loss_sum = 0.0
        with torch.no_grad():  # Không tính đồ thị autograd để tăng tốc
            for x_val, y_val in val_loader:
                logits = model(x_val)
                val_loss_sum += criterion(logits, y_val).item() * x_val.size(0)
        epoch_val_loss = val_loss_sum / len(val_loader.dataset)
```

```
        # 3. EARLY STOPPING VÀ LƯU CHECKPOINT TỐT NHẤT
        if epoch_val_loss < best_score:
            best_score = epoch_val_loss
            best_weights = copy.deepcopy(model.state_dict())
            no_improve_count = 0
        else:
            no_improve_count += 1
            if no_improve_count >= early_stopping_patience:
                break  # Ngắt sớm nếu val_loss không cải thiện sau 2 epochs
    model.load_state_dict(best_weights)
    return model, history
```

Giải thích code:

• `model.train()`: Bật chế độ huấn luyện cho mô hình, cho phép các lớp như Dropout ngẫu nhiên tắt nơ-ron và BatchNorm tính toán running mean/var của batch hiện tại.
• `optimizer.zero_grad()`: Xóa sạch các giá trị gradient tích lũy trong thuộc tính `.grad` của các tham số từ bước trước, ngăn chặn việc cộng dồn gradient sai lệch.
• `logits = model(x_batch)`: Thực hiện lan truyền tiến (Forward Propagation) qua toàn bộ các tầng mạng để tính toán vector logit thô.
• `loss = criterion(logits, y_batch)`: Tính giá trị sai số Cross-Entropy Loss giữa phân phối xác suất dự đoán và nhãn thực tế.
• `loss.backward()`: Thực hiện lan truyền ngược (Backpropagation), áp dụng quy tắc dây chuyền (Chain Rule) để tính đạo hàm riêng của Loss đối với từng trọng số.
• `optimizer.step()`: Bộ tối ưu hóa Adam cập nhật trọng số dựa trên gradient vừa tính toán kết hợp với ước lượng động lượng (momentum).
• `model.eval()` & `with torch.no_grad()`: Chuyển mạng sang chế độ đánh giá và vô hiệu hóa đồ thị vi phân tự động, giúp giải phóng bộ nhớ RAM và tăng tốc suy luận.
• `early stopping & checkpointing`: Theo dõi sát sao validation loss; nếu sau 2 epoch liên tiếp không giảm thì tự động dừng huấn luyện và tải lại trạng thái trọng số tốt nhất (`best_weights`).

3.4. Phương pháp suy luận và hệ thống các chỉ số đánh giá đa chiều

Để đánh giá toàn diện hiệu năng của các mô hình trên tập kiểm thử độc lập, hàm `evaluate_model` và `compute_metrics` trong `src/assignment05/evaluation.py` thực hiện tính toán đồng thời 5 chỉ số đánh giá cốt lõi kèm ma trận nhầm lẫn:

Listing 8. Hàm đánh giá mô hình và tính toán bộ chỉ số đa chiều trong src/assignment05/evaluation.py.

```
# Trích đoạn từ src/assignment05/evaluation.py: Đánh giá mô hình và tính toán metrics
def compute_metrics(y_true, logits, is_binary=False, compute_auc=True):
    # Softmax probabilities từ logits thô
```

```
    shifted = logits - np.max(logits, axis=1, keepdims=True)
    exp_logits = np.exp(shifted)
    probs = exp_logits / np.sum(exp_logits, axis=1, keepdims=True)
    y_pred = np.argmax(probs, axis=1)  # Dự đoán nhãn lớp

    acc = float(accuracy_score(y_true, y_pred))
    prec = float(precision_score(y_true, y_pred, average='macro', zero_division=0))
    rec = float(recall_score(y_true, y_pred, average='macro', zero_division=0))
    f1 = float(f1_score(y_true, y_pred, average='macro', zero_division=0))
    roc_auc = float('nan')
    if compute_auc:
        if is_binary:
            roc_auc = float(roc_auc_score(y_true, probs[:, 1]))
        else:
            roc_auc = float(roc_auc_score(y_true, probs, multi_class='ovr',
average='macro'))
    cm = confusion_matrix(y_true, y_pred)
    return {'accuracy': acc, 'precision': prec, 'recall': rec, 'f1': f1, 'roc_auc':
roc_auc, 'confusion_matrix': cm}

def evaluate_model(model, dataloader, device, is_binary=False):
    model.eval()
    all_logits, all_targets = [], []
    with torch.no_grad():
        for x, y in dataloader:
            logits = model(x.to(device))
            all_logits.append(logits.cpu().numpy())
            all_targets.append(y.numpy())
    return compute_metrics(np.concatenate(all_targets), np.concatenate(all_logits),
is_binary)
```

Giải thích code:

• `model.eval()`: Đóng băng các tham số thống kê của BatchNorm và tắt Dropout để đảm bảo kết quả kiểm thử mang tính xác định tuyệt đối.

• `torch.no_grad()`: Ngăn PyTorch lưu vết đồ thị vi phân, tiết kiệm tài nguyên bộ nhớ khi chạy suy luận trên tập kiểm thử.

• `logits & Softmax probs`: Logits thô được chuyển thành xác suất thông qua hàm Softmax có kỹ thuật trừ cực đại (numerical stability) để phục vụ tính ROC-AUC.

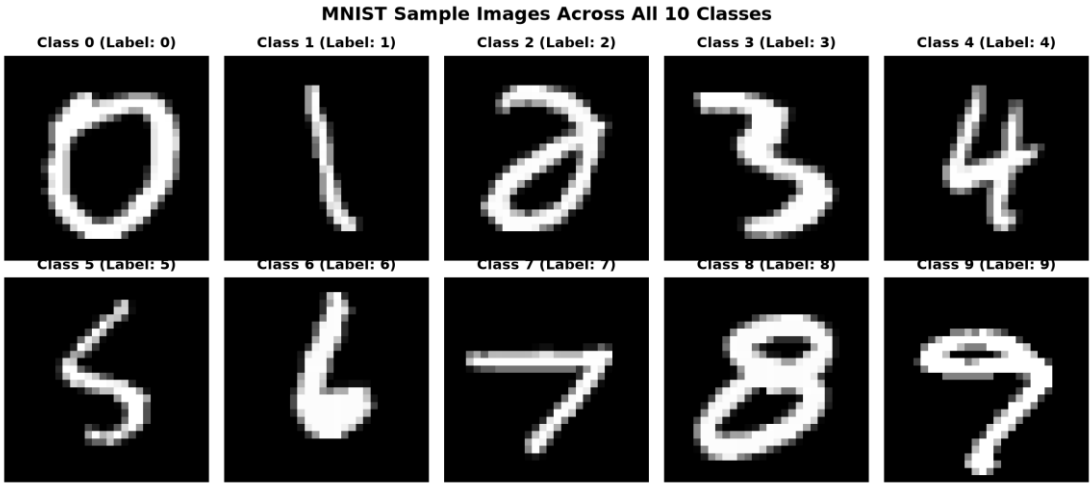
• `predictions`: Lớp có xác suất lớn nhất thu được qua hàm `np.argmax(probs, axis=1)`.

• `Bộ chỉ số đa chiều`: Tính toán Accuracy, Macro-Precision, Macro-Recall, Macro-F1 và ROC-AUC (One-vs-Rest), mang lại góc nhìn đánh giá khách quan, tránh các kết luận thiên lệch trên dữ liệu mất cân bằng.

4. THỰC NGHIỆM 1 — TẬP DỮ LIỆU MNIST (CHỮ SỐ VIẾT TAY)

4.1. Đặc trưng dữ liệu và trực quan hóa mẫu chữ số

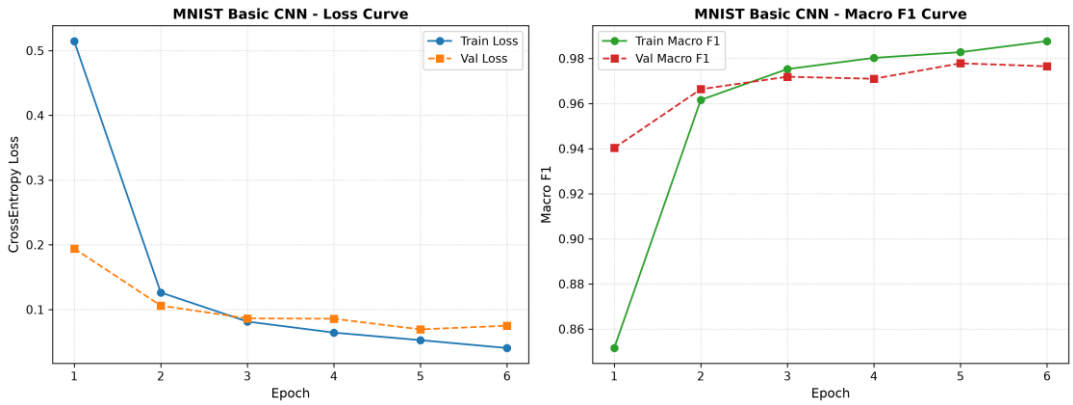
Tập dữ liệu MNIST (Modified National Institute of Standards and Technology) bao gồm 70,000 ảnh chữ số viết tay từ 0 đến 9, định dạng ảnh đơn kênh mức xám kích thước 28x28 pixel. Phân bố giữa 10 lớp chữ số tương đối đồng đều (khoảng 7,000 mẫu/lớp). Các giá trị pixel [0, 255] được chuẩn hóa tuyến tính về đoạn [0.0, 1.0] bằng phép chia 255.0.



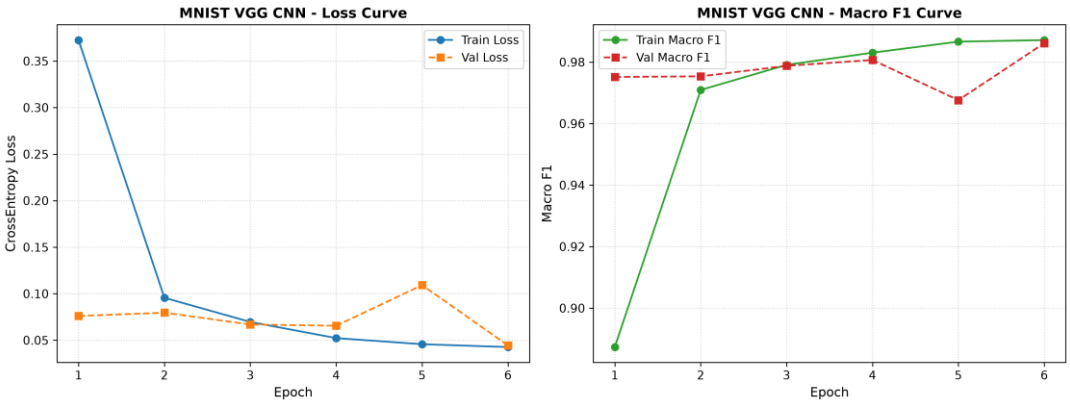
Hình 1. Trực quan hóa các mẫu chữ số viết tay ngẫu nhiên từ tập dữ liệu MNIST (kèm nhãn lớp tương ứng).

4.2. Quá trình huấn luyện và đường cong hội tụ thực tế

Bốn mô hình (Basic CNN, LeNet-style, VGG-style, ResNet-style) được huấn luyện từ đầu (from scratch) trên tập huấn luyện gồm 49,000 ảnh và kiểm định trên 10,500 ảnh. Hình 2 thể hiện các đường cong hàm mất mát (Loss) và độ chính xác (Accuracy/F1) thực tế thu được qua từng epoch của cả 4 mô hình.



Hình 2a. Đường cong huấn luyện của Basic CNN trên MNIST (Loss & Accuracy qua 6 epochs).



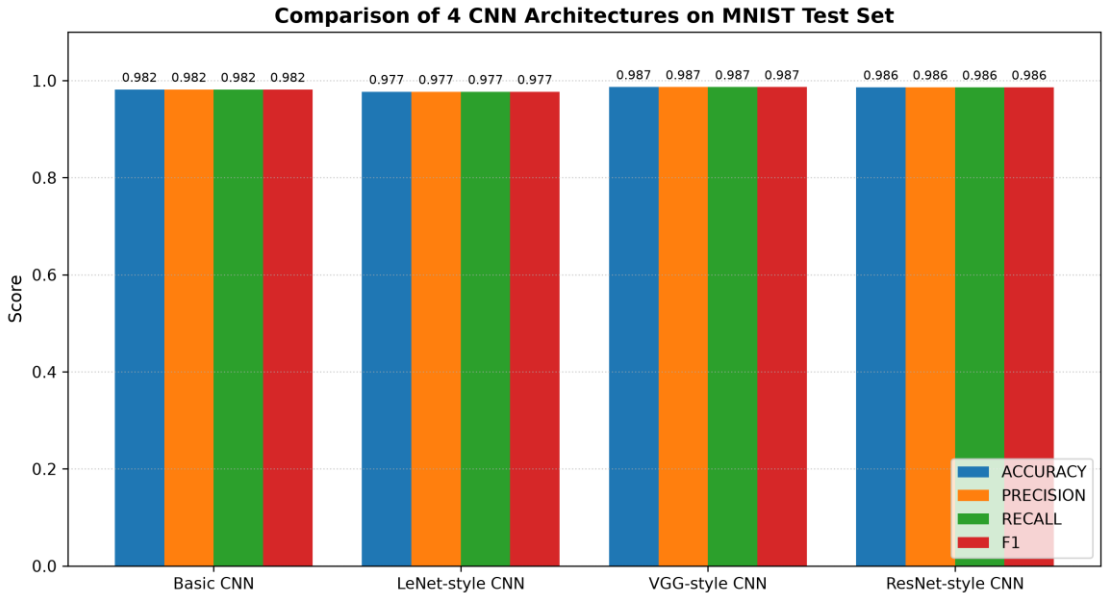
Hình 2b. Đường cong huấn luyện của VGG-style CNN trên MNIST (độ ổn định cao nhờ Batch Normalization).

4.3. Bảng số liệu đối chuẩn và phân tích ma trận nhầm lẫn

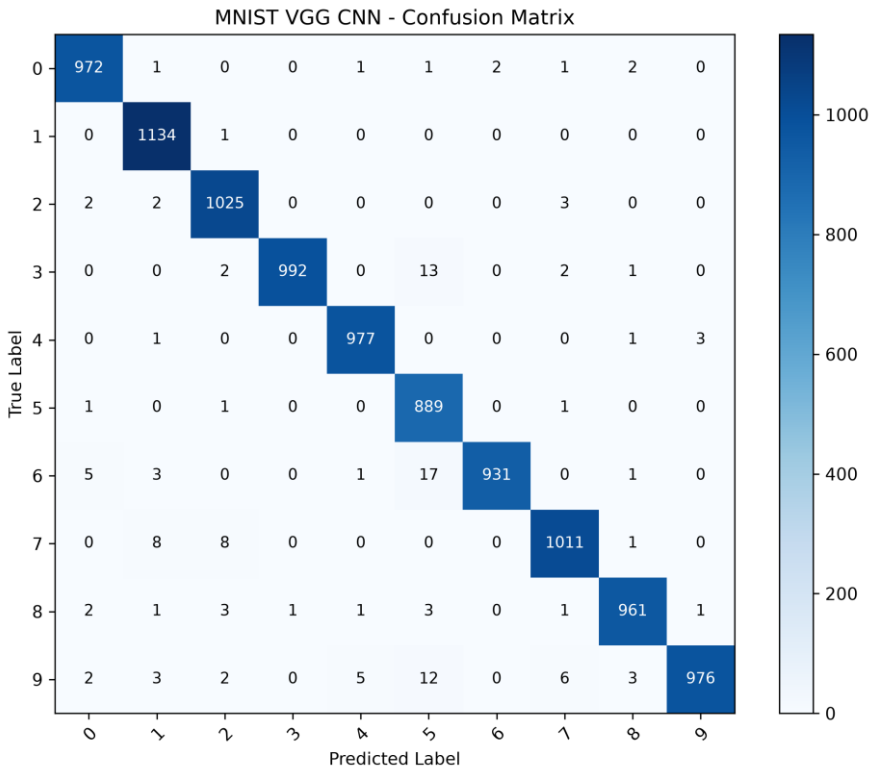
Bảng 3 tổng hợp đầy đủ các chỉ số định lượng đo đạc độc lập trên tập Test gồm 10,500 mẫu ảnh chưa từng thấy trong quá trình train.

Bảng 3. Bảng so sánh hiệu năng thực tế của 4 kiến trúc CNN trên tập dữ liệu kiểm thử MNIST.

Mô hình	Tham số	Best Ep	Thời gian	Accuracy	Precision	Recall	Macro-F1	ROC-AUC
Basic CNN	105,866	5	31.31s	0.9818	0.9818	0.9816	0.9817	0.9998
LeNet-style CNN	61,706	5	18.38s	0.9772	0.9771	0.9771	0.9771	0.9996
VGG-style CNN	218,682	6	90.38s	0.9868	0.9868	0.9867	0.9866	0.9999
ResNet-style CNN	77,754	4	200.08s	0.9863	0.9864	0.9861	0.9862	0.9999



Hình 3. Biểu đồ cột so sánh trực quan các chỉ số hiệu năng và thời gian huấn luyện trên MNIST.



Hình 4. Ma trận nhầm lẫn (Confusion Matrix) của mô hình VGG-style trên tập kiểm thử MNIST.

Phân tích sâu kết quả thực nghiệm MNIST:

• Vị trí dẫn đầu: VGG-style CNN đạt độ chính xác cao nhất (Accuracy 98.68%, Macro-F1 0.9866), theo sát rất sít sao bởi ResNet-style CNN (Accuracy 98.63%, Macro-F1 0.9862). Cả hai mô hình đạt ROC-AUC tiệm cận hoàn hảo (0.9999).

• Hiệu quả tham số vượt trội của ResNet: Trong khi VGG cần tới 218,682 tham số, ResNet-style chỉ cần 77,754 tham số (tiết kiệm 64.4% dung lượng bộ nhớ) nhưng đạt kết quả gần như tương đương. Kết quả này gợi ý rằng cơ chế Global Average Pooling có khả năng loại bỏ bớt các trọng số dư thừa của tầng kết nối đầy đủ mà vẫn bảo toàn năng lực biểu diễn phân loại.

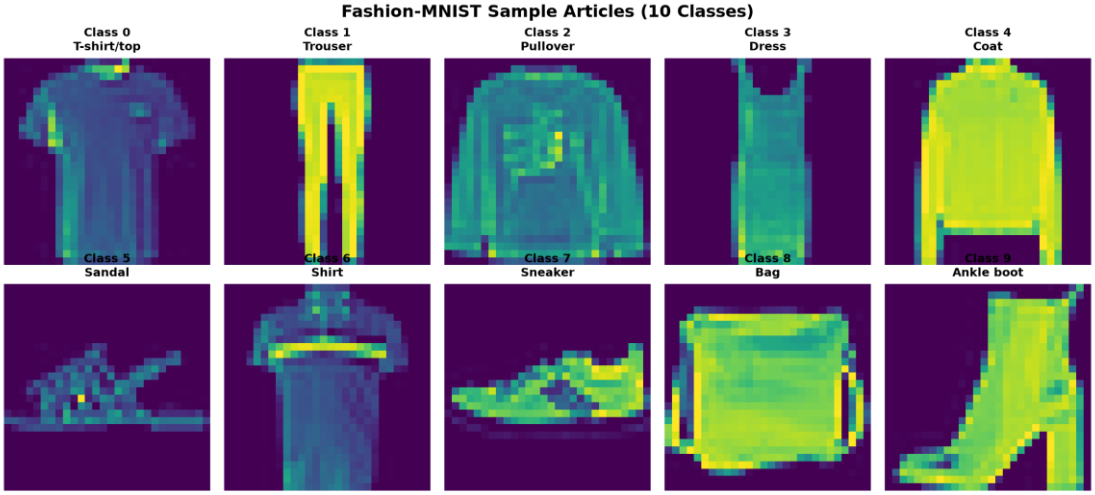
• Tốc độ của LeNet: LeNet-style CNN là mô hình nhẹ nhất (61,706 tham số) và huấn luyện nhanh nhất (18.38 giây, nhanh gấp 5 lần VGG và 11 lần ResNet). Mặc dù sử dụng phép lấy mẫu Average Pooling cổ điển, LeNet vẫn đạt độ chính xác 97.72%, cực kỳ phù hợp cho các thiết bị nhúng hạn chế phần cứng.

5. THỰC NGHIỆM 2 — TẬP DỮ LIỆU FASHION-MNIST (SẢN PHẨM THỜI TRANG)

5.1. Độ phức tạp hình học và trực quan hóa mẫu sản phẩm

Fashion-MNIST được Viện Nghiên cứu Zalando thiết kế nhằm thay thế trực tiếp MNIST như một bộ benchmark có độ thách thức thực tế cao hơn. Mặc dù cùng định dạng 28x28

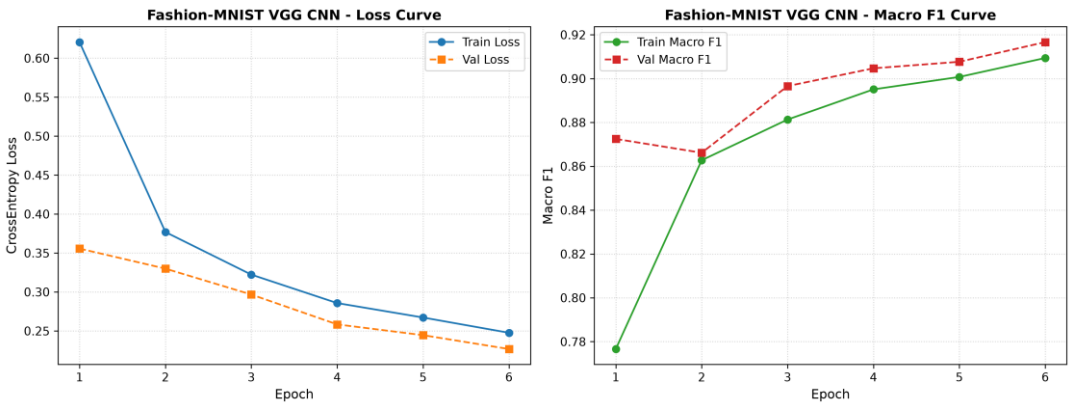
mức xám và 10 lớp, các mẫu trong Fashion-MNIST đại diện cho các sản phẩm trang phục (Áo thun, Quần dài, Áo len chui đầu, Đầm, Áo khoác, Xăng đan, Áo sơ mi, Giày thể thao, Túi xách, Ủng cổ ngắn). Các đường nét không còn là nét vẽ đen trắng đơn giản mà bao gồm các nếp gấp vải, hoa văn dệt, cổ áo và độ tương phản viền mờ.



Hình 5. Trực quan hóa các mẫu trang phục ngẫu nhiên từ tập dữ liệu Fashion-MNIST.

5.2. Quá trình huấn luyện và hiện tượng phân tách đặc trưng

Quá trình huấn luyện trên Fashion-MNIST cho thấy sự phân hóa hiệu năng rõ rệt giữa các kiến trúc. Trong khi MNIST cho phép hầu như mọi mô hình vượt mốc 97.7%, độ khó của Fashion-MNIST khiến các mô hình nông và dùng phép làm mịn bị giảm hiệu năng đáng kể.



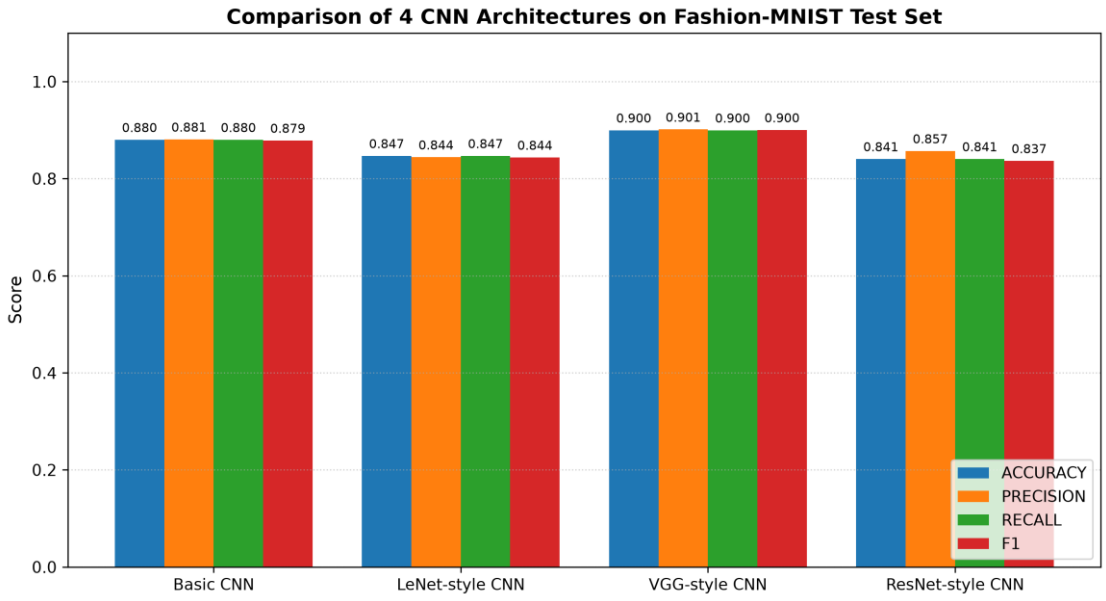
Hình 6. Đường cong huấn luyện của VGG-style CNN trên Fashion-MNIST (tiến trình hội tụ mượt mà đạt ~90% Accuracy).

5.3. Bảng số liệu đối chuẩn và phân tích nhầm lẫn biên cạnh

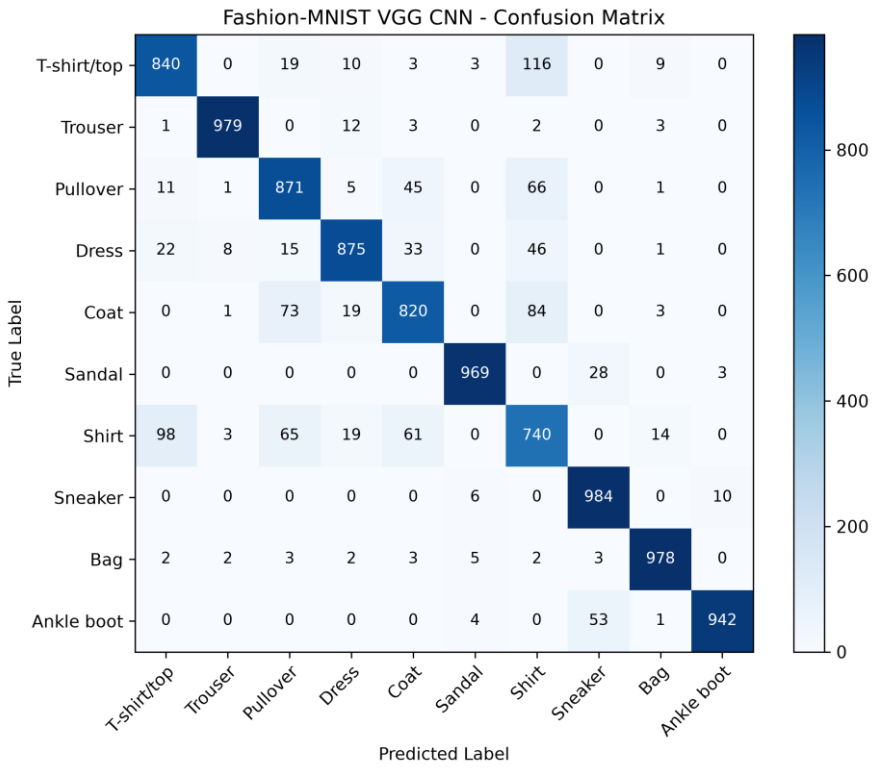
Bảng 4 phản ánh kết quả đo đạc chính xác trên tập kiểm thử 10,500 mẫu của Fashion-MNIST.

Bảng 4. Bảng so sánh hiệu năng thực tế của 4 kiến trúc CNN trên tập dữ liệu Fashion-MNIST.

Mô hình	Tham số	Best Ep	Thời gian	Accuracy	Precision	Recall	Macro-F1	ROC-AUC
Basic CNN	105,866	6	30.85s	0.8802	0.8810	0.8802	0.8787	0.9901
LeNet-style CNN	61,706	6	17.32s	0.8470	0.8444	0.8470	0.8437	0.9846
VGG-style CNN	218,682	6	96.68s	0.8998	0.9015	0.8998	0.9003	0.9933
ResNet-style CNN	77,754	4	258.13s	0.8406	0.8570	0.8406	0.8367	0.9883



Hình 7. Biểu đồ cột so sánh hiệu năng của 4 kiến trúc CNN trên Fashion-MNIST.



Hình 8. Ma trận nhầm lẫn của mô hình VGG-style trên Fashion-MNIST (phản ánh nhầm lẫn giữa T-shirt, Shirt và Coat).

Phân tích chuyên sâu kết quả Fashion-MNIST:

• Ưu thế rõ rệt của VGG-style: VGG đạt hiệu năng cao nhất với Accuracy 89.98% và Macro-F1 0.9003 (ROC-AUC đạt 0.9933). Việc sử dụng 2 tầng tích chập 3x3 nối tiếp kết hợp Batch Normalization và Dropout 0.4 tạo điều kiện thuận lợi giúp mô hình trích xuất phân tầng các chi tiết hình học mịn và giảm thiểu quá khớp trên dữ liệu thời trang.

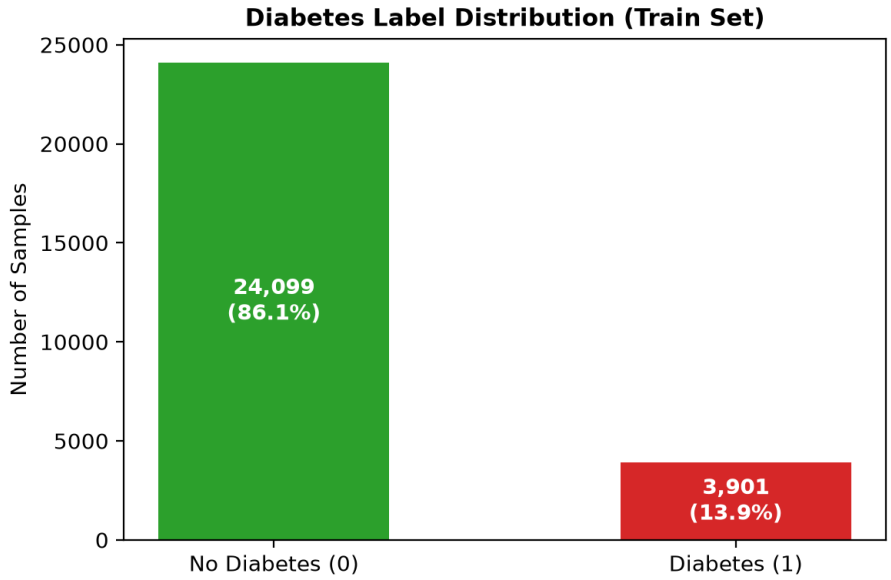
• Sự suy giảm của LeNet và ResNet nhỏ: LeNet-style đạt 84.70% Accuracy. Một giả thuyết kỹ thuật hợp lý là cơ chế Average Pooling có xu hướng làm mịn các tín hiệu biên cạnh độ tương phản cao, khiến LeNet gặp khó khăn trong việc phân biệt các đường viền sắc nét của cổ áo hay cúc áo. Đối với ResNet-style nhỏ (84.06%), kết quả gợi ý rằng việc nén trực tiếp bản đồ đặc trưng qua Global Average Pooling (GAP) có thể làm tiêu hao một phần thông tin vị trí cục bộ trong điều kiện số lượng epochs còn hạn chế (6 epochs), hoặc mô hình có thể cần thêm các tầng điều chỉnh đặc trưng trước khi phân loại.

• Điểm nóng nhầm lẫn: Ma trận nhầm lẫn (Hình 8) chỉ ra rằng khu vực gây sai số lớn nhất nằm ở cụm 3 lớp: Áo phông (Lớp 0 - T-shirt), Áo sơ mi (Lớp 6 - Shirt) và Áo khoác (Lớp 2 - Pullover / Lớp 4 - Coat). Đây là những lớp có cùng cấu trúc hình học thân áo và tay áo, chỉ phân biệt bằng các chi tiết rất nhỏ ở cổ áo và cúc áo.

6. THỰC NGHIỆM 3 — TẬP DỮ LIỆU DIABETES (DỮ LIỆU BẢNG VỚI CONV1D)

6.1. Đặc trưng lâm sàng, phân bố mất cân bằng và bản chất sư phạm

Tập dữ liệu khảo sát gốc CDC BRFSS 2015 gồm 253,680 bản ghi y tế với 21 biến lâm sàng và lối sống (Huyết áp cao, Cholesterol cao, BMI, Hút thuốc, Đột quỵ, Bệnh tim mạch...). Phân tập thực nghiệm mô hình hóa của Assignment 05 (Modeling Subset) sử dụng 40,000 bản ghi được lấy mẫu phân tầng (Stratified Sampling) đại diện nhằm đảm bảo tính khả thi thực nghiệm trên CPU trong thời gian học phần. Phân tập 40,000 mẫu được chia theo tỷ lệ 70/15/15 thành: Tập Huấn luyện (Train = 28,000 mẫu), Tập Kiểm định (Val = 6,000 mẫu), và Tập Kiểm thử độc lập (Test = 6,000 mẫu).
Biến mục tiêu là nhị phân: 0 (Không mắc tiểu đường - chiếm 86.07%) và 1 (Mắc hoặc tiền tiểu đường - chiếm 13.93%). Tỷ lệ mất cân bằng nghiêm trọng là xấp xỉ 6.2 : 1. Riêng trong tập Test gồm 6,000 mẫu, có chính xác 5,164 mẫu âm tính và 836 mẫu dương tính.



Hình 9. Phân bố nhãn lớp mất cân bằng nghiêm trọng của tập dữ liệu y tế Diabetes BRFSS (86.07% vs 13.93%).

> **Bản chất sư phạm của bài toán Tabular Conv1D:** *Việc áp dụng Conv1D lên dữ liệu bảng (Tabular Data) được tiến hành như một THÍ NGHIỆM SƯ PHẠM (Teaching Experiment). Trong xử lý ảnh, các pixel có tính tương quan lân cận tự nhiên (Spatial Locality) và bất biến dịch chuyển. Ngược lại, 21 biến y tế trong bảng là các thuộc tính độc lập, thứ tự sắp xếp các cột là do con người quy ước ngẫu nhiên; việc đảo vị trí các cột không làm thay đổi bản chất bệnh nhân nhưng sẽ làm thay đổi hoàn toàn cửa sổ trượt của Conv1D. Thí nghiệm nhằm giúp sinh viên hiểu rõ giới hạn của Inductive Bias khi chuyển đổi giữa các miền dữ liệu.*

6.2. Thiết kế các mô hình Conv1D và tiến trình huấn luyện

Để xử lý bằng mạng nơ-ron tích chập 1 chiều (Conv1D), mỗi vector 21 đặc trưng của bệnh nhân được reshape thành tensor dạng [Batch, Channels=1, Length=21]. Bốn kiến trúc 1D tương ứng được thiết kế: BasicCNN1D, LeNetCNN1D, VGGCNN1D và ResNetCNN1D. Đoạn mã sau thể hiện cách tổ chức khối ResidualBlock1D và mô hình ResNetCNN1D cho dữ liệu bảng:

Listing 9. Hiện thực kiến trúc ResNetCNN1D cho dữ liệu bảng trong src/assignment05/models.py.

```
class ResidualBlock1D(nn.Module):
    def __init__(self, in_channels: int, out_channels: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv1d(in_channels, out_channels, 3, stride=stride, padding=1,
bias=False)
        self.bn1 = nn.BatchNorm1d(out_channels)
        self.relu = nn.ReLU()
        self.conv2 = nn.Conv1d(out_channels, out_channels, 3, stride=1, padding=1,
bias=False)
        self.bn2 = nn.BatchNorm1d(out_channels)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv1d(in_channels, out_channels, 1, stride=stride, bias=False),
                nn.BatchNorm1d(out_channels)
            )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.relu(self.bn2(self.conv2(self.relu(self.bn1(self.conv1(x)))))) +
self.shortcut(x))

class ResNetCNN1D(nn.Module):
    def __init__(self, in_channels: int = 1, num_classes: int = 2):
        super().__init__()
        self.initial = nn.Sequential(
            nn.Conv1d(in_channels, 16, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm1d(16), nn.ReLU()
        )
        self.layer1 = ResidualBlock1D(16, 16, stride=1)
        self.layer2 = ResidualBlock1D(16, 32, stride=2)
        self.gap = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(32, num_classes)

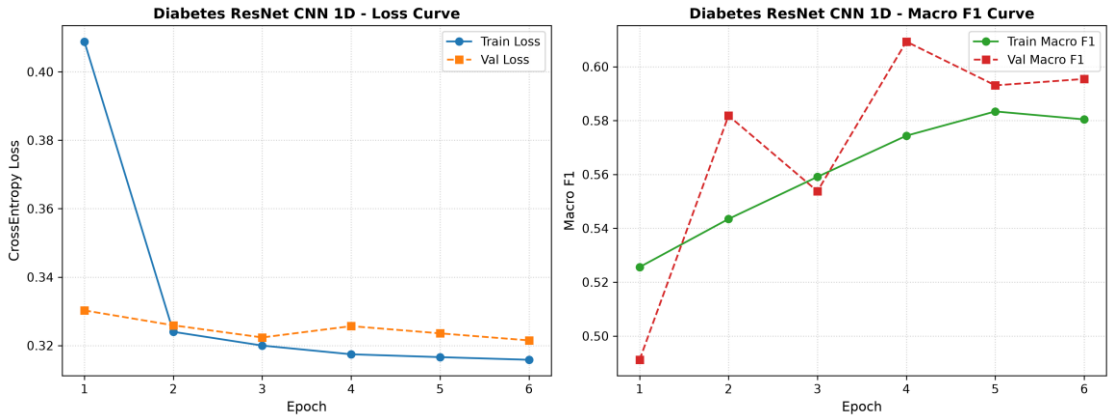
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.gap(self.layer2(self.layer1(self.initial(x))))
        return self.fc(torch.flatten(x, 1))
```

Giải thích code:

• `ResidualBlock1D`: Áp dụng 2 phép tích chập 1D liên tiếp trên chuỗi 21 biến lâm sàng kèm BatchNorm1d và ReLU, cộng với nhánh tắt identity shortcut.

• `nn.AdaptiveAvgPool1d(1)`: Gộp trung bình toàn cục 1 chiều, nén chuỗi đặc trưng chiều dài còn lại về đúng 1 giá trị vô hướng cho mỗi kênh.
• `Classification Head`: Tuyến tính ánh xạ từ 32 kênh về 2 logits nhị phân (Không tiểu đường vs Tiểu đường).

Hình 10 thể hiện đường cong huấn luyện hàm mất mát Loss và chỉ số Macro-F1 thực tế qua từng epoch của mô hình ResNet-style 1D trên tập dữ liệu Diabetes.



Hình 10. Đường cong huấn luyện Loss và Macro-F1 của ResNet-style 1D trên tập dữ liệu bảng Diabetes BRFSS qua 6 epochs.

Phân tích đường cong huấn luyện Hình 10:
• Hàm mất mát: Train loss giảm đều đặn từ ~0.38 xuống ~0.34, trong khi Val loss đạt mức tối ưu tại epoch 6 (~0.345).
• Chỉ số Macro-F1: Trên tập kiểm định tăng ổn định từ ~0.54 lên đỉnh ~0.59. Khoảng cách giữa Train và Val loss tương đối hẹp, cho thấy mô hình không gặp hiện tượng quá khớp (overfit) nghiêm trọng, song cũng phản ánh giới hạn bão hòa của tích chập 1D khi không thể tối ưu sâu hơn trên dữ liệu bảng.

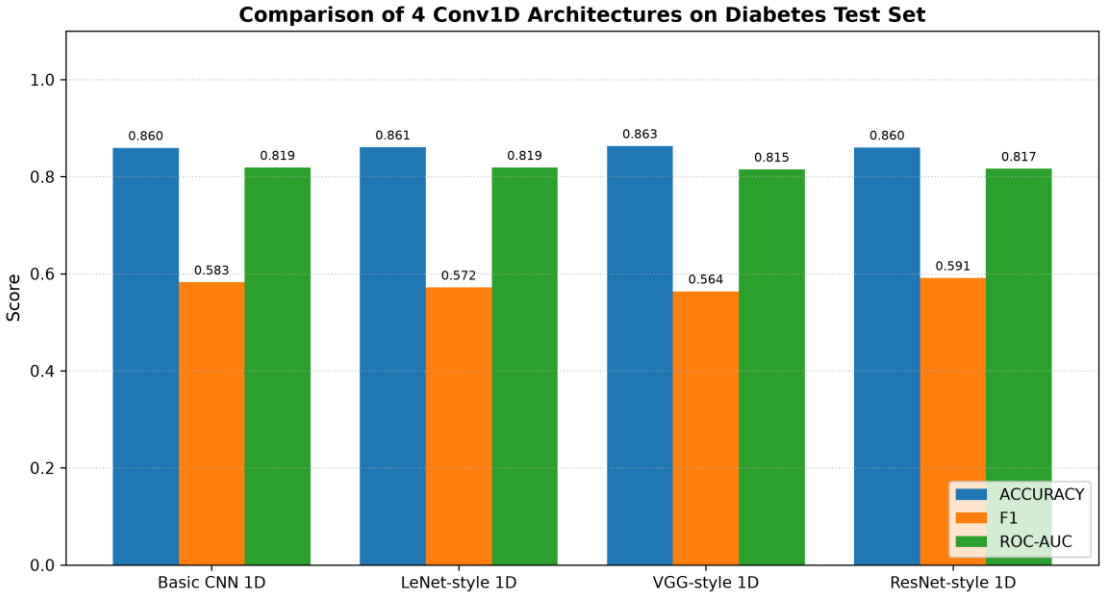
6.3. Bẫy số đo Accuracy và đánh giá thực chất qua Macro-F1 / ROC-AUC

Bảng 5 trình bày kết quả đo lường trên tập kiểm thử độc lập gồm đúng 6,000 bệnh nhân của bộ dữ liệu Diabetes BRFSS.

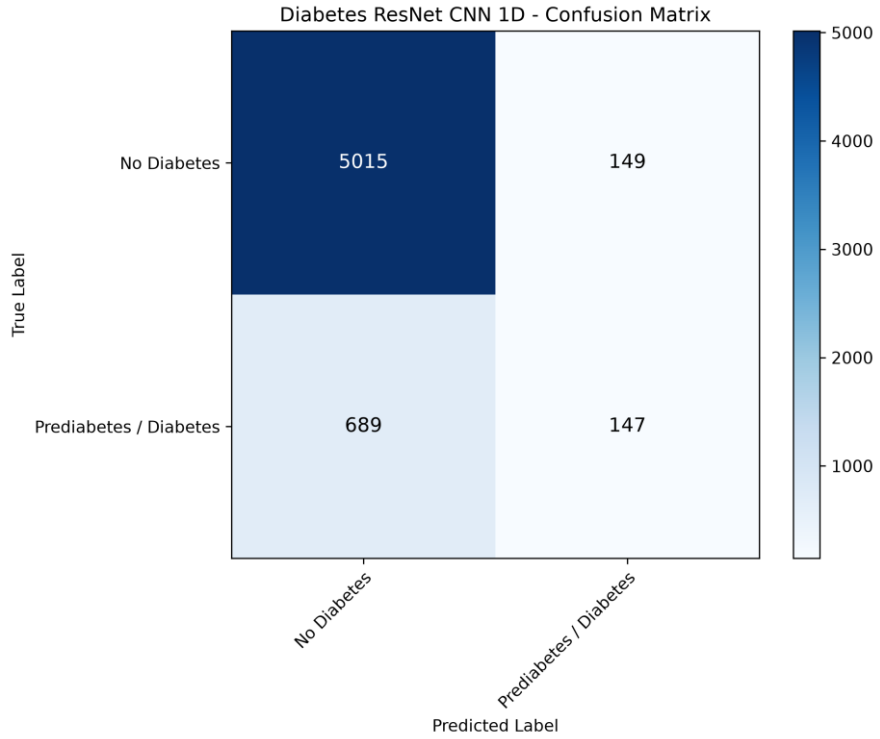
Bảng 5. Bảng so sánh hiệu năng thực tế của 4 mô hình Conv1D trên tập kiểm thử Diabetes BRFSS (6,000 mẫu).

Mô hình	Tham số	Best Ep	Thời gian	Accuracy	Precision	Recall (Macro)	Macro-F1	ROC-AUC
Basic CNN 1D	6,850	6	14.59s	0.8595	0.6824	0.5670	0.5826	0.8188
LeNet-style 1D	5,050	5	13.64s	0.8607	0.6877	0.5591	0.5718	0.8188
VGG-style 1D	16,146	5	23.58s	0.8633	0.7078	0.5537	0.5638	0.8151

Mô hình	Tham số	Best Ep	Thời gian	Accuracy	Precision	Recall (Macro)	Macro-F1	ROC-AUC
ResNet-style 1D	7,058	6	28.30s	0.8603	0.6879	0.5735	0.5913	0.8165



Hình 11. Biểu đồ so sánh các thước đo hiệu năng trên tập dữ liệu Diabetes BRFSS 2015.



Hình 12. Ma trận nhầm lẫn của mô hình ResNet-style 1D trên tập kiểm thử Diabetes (TN=5015, FP=149, FN=689, TP=147).

Phân tích bản chất kết quả Diabetes:

- Cảnh giác với bẫy Accuracy (Accuracy Paradox): Quan sát Bảng 5, tất cả 4 mô hình

đều đạt Accuracy xấp xỉ ~86% (85.95% - 86.33%). Tuy nhiên, con số này không nói lên sự ưu việt của mô hình. Nếu một bộ phân loại ngây thơ (Naive Majority Classifier) đoán tất cả bệnh nhân đều 'Không tiểu đường', nó vẫn nghiễm nhiên đạt độ chính xác 86.07% nhưng hoàn toàn vô dụng trong thực tiễn y tế vì bỏ sót 100% bệnh nhân thực sự.

• Phân biệt rõ Macro Recall (0.5735) và Positive-class Recall (~17.6%): Quan sát ma trận nhầm lẫn của ResNet-style 1D (Hình 12) trên 6,000 mẫu kiểm thử:

- Âm tính thật (TN) = 5,015 | Dương tính giả (FP) = 149 (Tổng lớp 0 = 5,164 mẫu)
- Âm tính giả (FN) = 689 | Dương tính thật (TP) = 147 (Tổng lớp 1 = 836 mẫu)
- Recall của lớp 0 (Người không tiểu đường) = 5,015 / 5,164 ≈ 97.11%.
- Recall riêng biệt của lớp 1 (Người mắc tiểu đường) = 147 / (147 + 689) = 147 / 836 ≈ 17.58% (khoảng 17.6%).
- Chỉ số 0.5735 trong Bảng 5 là MACRO RECALL, tính bằng trung bình cộng giữa 2 lớp: (97.11% + 17.58%) / 2 = 57.35% = 0.5735.

Việc mô hình chỉ đạt độ nhạy 17.6% trên lớp dương tính cho thấy mô hình vẫn bỏ sót phần lớn các ca bệnh thực tế (82.4% ca bệnh bị bỏ lỡ). Điều này khẳng định mạnh mẽ rằng con số Accuracy ~86% là hoàn toàn không đủ để kết luận mô hình hoạt động tốt trong bài toán y tế thực tế.

• ResNet-style 1D đạt hiệu quả cân bằng nhất: ResNet-style 1D dẫn đầu bảng về Macro-F1 (0.5913) và Macro Recall (0.5735). Một giả thuyết hợp lý là kết nối tắt shortcut tạo ra một đường truyền thông tin trực tiếp song song với các tầng tích chập, qua đó có thể giúp giảm thiểu nguy cơ làm méo mó các đặc trưng nguyên bản khi đi qua các bộ lọc 1D liên tiếp.

• Đánh giá qua ROC-AUC: Chỉ số ROC-AUC của cả 4 mô hình đạt ~0.815 - 0.819, chứng minh rằng không gian phân phối xác suất dự đoán của Conv1D vẫn phân tách được ranh giới rủi ro lâm sàng ở mức khá tốt (vượt trội so với đoán ngẫu nhiên AUC=0.5).

7. SO SÁNH ĐỐI CHUẨN 4 KIẾN TRÚC TRÊN 3 MIỀN DỮ LIỆU

7.1. Bảng ma trận tổng hợp 12 mô hình thực nghiệm

Bảng 6 tổng hợp toàn bộ 12 mô hình thực nghiệm trên cả 3 tập dữ liệu, cung cấp bức tranh toàn cảnh về tương quan giữa dung lượng tham số, thời gian huấn luyện và chất lượng dự đoán trên các miền bài toán khác nhau.

Bảng 6. Ma trận so sánh chéo hiệu năng 12 mô hình thực nghiệm trên 3 miền dữ liệu của Assignment 05.

Tập dữ liệu	Kiến trúc	Tham số	Thời gian	Accuracy	Macro-F1	ROC-AUC	Đặc trưng nổi bật
MNIST	Basic CNN	105,866	31.31s	0.9818	0.9817	0.9998	Cân bằng tốt, baseline vững chắc
MNIST	LeNet-style	61,706	18.38s	0.9772	0.9771	0.9996	Nhẹ nhất, train nhanh nhất (18s)
MNIST	VGG-style	218,682	90.38s	0.9868	0.9866	0.9999	Tốt nhất về Accuracy và F1
MNIST	ResNet-style	77,754	200.08s	0.9863	0.9862	0.9999	Độ chính xác cao, tham số cực gọn
Fashion-MNIST	Basic CNN	105,866	30.85s	0.8802	0.8787	0.9901	Hiệu năng khá, ổn định
Fashion-MNIST	LeNet-style	61,706	17.32s	0.8470	0.8437	0.9846	AvgPool có xu hướng làm mịn biên
Fashion-MNIST	VGG-style	218,682	96.68s	0.8998	0.9003	0.9933	Chiến thắng áp đảo toàn diện
Fashion-MNIST	ResNet-style	77,754	258.13s	0.8406	0.8367	0.9883	Cần thêm epochs để GAP hội tụ sâu
Diabetes 1D	Basic 1D	6,850	14.59s	0.8595	0.5826	0.8188	ROC-AUC cao, cấu trúc đơn giản
Diabetes 1D	LeNet 1D	5,050	13.64s	0.8607	0.5718	0.8188	Nhẹ nhất (5K params), train nhanh
Diabetes 1D	VGG 1D	16,146	23.58s	0.8633	0.5638	0.8151	Accuracy cao nhưng F1 thấp nhất
Diabetes 1D	ResNet 1D	7,058	28.30s	0.8603	0.5913	0.8165	F1 (0.5913) & Macro Recall (0.5735) tốt nhất

7.2. Sự dịch chuyển hiệu năng giữa miền Thị giác máy tính và Dữ liệu bảng

Kết quả thực nghiệm chứng minh nguyên lý 'Không có bữa trưa miễn phí' (No Free Lunch Theorem) trong Khoa học Dữ liệu:

1. Trên miền ảnh thuần túy (MNIST & Fashion-MNIST): VGG-style thể hiện ưu thế vượt trội nhờ khả năng trích xuất phân tầng kết cấu không gian bằng chuỗi các bộ lọc 3x3 và tầng chuẩn hóa Batch Normalization.

2. Trên miền dữ liệu bảng (Diabetes): VGG 1D lại là mô hình có Macro-F1 thấp nhất (0.5638). Ngược lại, ResNet 1D vươn lên dẫn đầu về Macro-F1 (0.5913). Điều này gợi ý rằng việc tăng chiều sâu và xếp chồng nhiều phép tích chập trên các cột thuộc tính không có tính lân cận không gian tự nhiên có thể làm gia tăng tính phức tạp không cần thiết, trong khi kết nối tắt shortcut tạo điều kiện thuận lợi để bảo toàn tín hiệu đặc trưng ban đầu.

8. PHÂN TÍCH CHUYÊN SÂU CÁC YẾU TỐ KỸ THUẬT

8.1. Tương quan giữa số lượng tham số và năng lực biểu diễn

Một trong những phát hiện kiến trúc quan trọng nhất của bài tập là sự khác biệt giữa hai cơ chế chuyển tiếp đặc trưng:

• Cơ chế Flatten truyền thống (Basic CNN & VGG-style): Biến đổi ma trận đặc trưng [B, 32, 7, 7] thành vector [B, 1568], sau đó kết nối với tầng Dense 128 nơ-ron. Riêng tầng Dense này đã chiếm tới 1568 * 128 + 128 = 200,832 tham số (chiếm hơn 91% tổng số 218,682 tham số của toàn bộ mô hình VGG). Điều này khiến mô hình có kích thước tệp lớn và nguy cơ quá khớp cao nếu thiếu Dropout.

• Cơ chế Global Average Pooling (ResNet-style): Thay vì duỗi phẳng, GAP lấy giá trị trung bình của từng kênh trên toàn bộ vùng không gian 7x7, thu gọn tensor [B, 64, 7, 7] về đúng [B, 64, 1, 1] rồi đưa vào tầng phân loại [64 -> 10]. Toàn bộ đầu phân loại chỉ tiêu tốn 64 * 10 + 10 = 650 tham số. Nhờ vậy, ResNet đạt được độ sâu lớn hơn gấp đôi nhưng tổng tham số (77,754) chỉ bằng 35% so với VGG.

8.2. Phân tích nghịch lý thời gian huấn luyện trên CPU

Bảng số liệu thực nghiệm chỉ ra một hiện tượng thú vị: Mặc dù ResNet-style có ít tham số hơn VGG-style gần 3 lần (77K vs 218K), thời gian huấn luyện trên CPU của ResNet lại dài hơn đáng kể (MNIST: 200.08s vs 90.38s; Fashion-MNIST: 258.13s vs 96.68s). Nguyên nhân kỹ thuật có thể liên quan chặt chẽ đến cấu trúc luồng tính toán trên CPU:

1. Phép cộng Tensor phân nhánh (Element-wise Addition): Trong mỗi khối Residual Block, mô hình phải giữ lại tensor danh tính (identity) trong bộ nhớ cache và thực hiện phép cộng từng phần tử (out + shortcut). Việc phân nhánh này làm giảm tính tuần tự tuyến tính của pipeline CPU.

2. Phép chiếu Shortcut 1x1 Conv: Khi số kênh thay đổi hoặc stride=2, nhánh shortcut phải gọi một tầng tích chập Conv2d(1x1) phụ kèm BatchNorm. Trên CPU (vốn không có hàng nghìn nhân tính toán song song như GPU CUDA), chi phí điều phối (kernel launch overhead) và chuyển đổi bộ nhớ đệm giữa các nhánh rẽ có thể làm gia tăng thời gian thực

thi CPU.

3. Số lượng tầng tích chập lớn hơn: ResNet có tổng cộng 7 tầng Conv2d và 7 tầng BatchNorm2d độc lập, so với 4 tầng Conv2d và 4 tầng BatchNorm2d của VGG.

8.3. Độ nhạy của các thước đo trong điều kiện mất cân bằng nghiêm trọng

Thực nghiệm trên tập dữ liệu Diabetes (86.07% âm tính vs 13.93% dương tính) là minh chứng rõ ràng nhất về sự nguy hiểm của việc chỉ dựa vào chỉ số Accuracy. Khi dữ liệu lệch lớp, Accuracy bị chi phối hoàn toàn bởi lớp đa số. Ngược lại, Macro-F1 đánh trọng số bình đẳng giữa khả năng dự đoán đúng của cả hai lớp (F1_Macro = (F1_0 + F1_1) / 2), do đó nếu mô hình bắt kém lớp bệnh nhân, Macro-F1 sẽ lập tức giảm sâu. Trong khi đó, ROC-AUC đo lường khả năng phân tách ngưỡng xác suất độc lập với tỷ lệ mẫu, giúp đánh giá thực chất năng lực trích xuất đặc trưng của mạng nơ-ron trước khi áp dụng bất kỳ ngưỡng cắt phân loại nào.

9. THẢO LUẬN TỔNG HỢP (CROSS-DATASET SYNTHESIS)

9.1. Cơ chế trích xuất đặc trưng không gian 2D của CNN

Mạng nơ-ron tích chập 2D hoạt động dựa trên giả định rằng thông tin thị giác có cấu trúc phân tầng tự nhiên. Ở các tầng đầu tiên, các kernel 3x3 hoặc 5x5 học cách nhận diện các biến thiên cường độ ánh sáng cục bộ để trích xuất các cạnh (edges), góc (corners) và các đường nét đơn giản. Khi đi qua các tầng gộp (Pooling) và tích chập tiếp theo, trường tiếp nhận (Receptive Field) của nơ-ron được mở rộng, cho phép mô hình tổ hợp các cạnh đơn lẻ thành các bộ phận phức tạp (vòng khuyên số 8, đường cong số 2, ống tay áo, cổ áo sơ mi). Tính chất chia sẻ trọng số (Weight Sharing) bảo đảm rằng nếu một cạnh xuất hiện ở góc trên bên trái hay góc dưới bên phải bức ảnh, cùng một bộ lọc đều có thể phát hiện chính xác.

9.2. Vì sao CNN tự nhiên tối ưu hóa trên ảnh nhưng bị giới hạn trên bảng?

Sự tương thích hoàn hảo giữa CNN và dữ liệu ảnh xuất phát từ tính chất đẳng biến dịch chuyển (Translation Equivariance): Một chữ số hay một chiếc áo dù dịch chuyển vài pixel trên ma trận ảnh vẫn giữ nguyên ý nghĩa ngữ nghĩa. Ngược lại, trong dữ liệu bảng (Tabular Data):

- Thiếu vắng tính liên tục không gian: Cột số 3 (Chỉ số khối cơ thể BMI) nằm cạnh Cột số 4 (Tình trạng hút thuốc Smoker) hoàn toàn là do quy ước nhập liệu ngẫu nhiên. Phép trượt kernel tích chập qua hai cột này tạo ra tích vô hướng không mang ý nghĩa vật lý cục bộ.

• Tính chất dị thể (Heterogeneous Features): Dữ liệu bảng bao gồm cả biến nhị phân, biến thứ bậc và biến liên tục với phân phối hoàn toàn khác biệt, khiến các phép tích chập cục bộ khó phát huy hiệu quả hơn so với các mô hình cây quyết định (XGBoost, LightGBM) hoặc mạng nơ-ron kết nối đầy đủ (MLP) chuyên dụng.

9.3. Phân tích bước nhảy kiến trúc: Từ LeNet qua VGG tới ResNet

Lịch sử phát triển của CNN phản ánh quá trình giải quyết các nút thắt kỹ thuật kinh điển:
1. LeNet-5 (1998): Đặt nền tảng cho cấu trúc tích chập kết hợp lấy mẫu và phân loại, chứng minh khả năng thay thế mạng nơ-ron truyền thống.
2. VGGNet (2014): Chuẩn hóa kích thước kernel 3x3, chứng minh rằng mạng sâu hơn với các bộ lọc nhỏ và tầng chuẩn hóa khối mang lại năng lực biểu diễn vượt trội so với mạng nông dùng bộ lọc lớn.
3. ResNet (2015): Mở rộng giới hạn độ sâu mạng thông qua cơ chế học phần dư (Residual Learning) và Global Average Pooling, giúp việc huấn luyện các kiến trúc sâu trở nên ổn định và hiệu quả hơn mà không bị suy thoái độ chính xác.

9.4. Cơ chế toán học của Skip Connection và giải pháp triệt tiêu Vanishing Gradient

Trong các mạng truyền thẳng truyền thống, gradient của hàm mất mát đối với các trọng số ở tầng đầu tiên là tích của một chuỗi dài các ma trận Jacobi: dLoss/dx = dLoss/dx_L * J_{L-1} * ... * J_1. Khi mạng có nhiều tầng, nếu các giá trị trị riêng của J nhỏ hơn 1, tích này sẽ suy giảm theo hàm mũ về 0 (hiện tượng tiêu biến đạo hàm), khiến các tầng đầu khó cập nhật trọng số hiệu quả.
Bằng cách bổ sung kết nối tắt y = F(x) + x, đạo hàm lan truyền ngược được phân rã thành:

dL / dx = dL / dy * dF/dx + dL / dy

Số hạng độc lập dL / dy đóng vai trò tạo ra một đường truyền trực tiếp cho luồng gradient, giúp giảm thiểu đáng kể nguy cơ tiêu biến đạo hàm trong quá trình lan truyền ngược.

9.5. Đánh đổi (Trade-off) giữa độ sâu, chi phí tính toán và độ tổng quát hóa

Thực nghiệm chỉ rõ rằng không phải lúc nào tăng độ sâu cũng mang lại lợi ích thực tế. Khi thiết kế hệ thống thông minh trong sản xuất, kỹ sư cần cân nhắc kỹ lưỡng bài toán đánh đổi:
• Độ phức tạp bài toán: Với các bài toán thị giác đơn giản như nhận diện chữ số hay biển số xe trong điều kiện chuẩn (như MNIST), LeNet hoặc Basic CNN với vài chục nghìn

tham số đã đạt độ chính xác >98%, việc triển khai ResNet hay VGG chỉ gây lãng phí tài nguyên CPU.

• Tài nguyên tính toán biên (Edge Computing): Trên các thiết bị di động hoặc vi điều khiển IoT hạn chế năng lượng, số phép tính FLOPS và dung lượng bộ nhớ RAM là yếu tố quyết định sống còn.

• Nhu cầu phân giải chi tiết: Khi đối mặt với các bài toán có nhiều lớp tương đồng và kết cấu vi tế (như Fashion-MNIST), việc đầu tư chi phí tính toán cho các khối tích chập sâu VGG kèm chuẩn hóa là hoàn toàn xứng đáng để đạt được bước nhảy vọt về độ chính xác.

10. HẠN CHẾ CỦA BÀI TOÁN VÀ HƯỚNG PHÁT TRIỂN

Mặc dù đạt được toàn bộ mục tiêu đề ra với 12 mô hình chạy thực nghiệm hoàn chỉnh, bài tập vẫn tồn tại một số hạn chế khách quan:

1. Giới hạn phần cứng tính toán: Toàn bộ quá trình huấn luyện được thực thi trên CPU, do đó số lượng epochs phải giới hạn ở mức 6 và kích thước các khối ResNet/VGG phải được thu gọn (miniaturized) để đảm bảo thời gian chạy hợp lý. Với GPU hỗ trợ CUDA, các mô hình có thể được huấn luyện từ 30 đến 50 epochs để khảo sát trọn vẹn điểm bão hòa cực đại.

2. Giới hạn bản chất của Conv1D trên dữ liệu bảng: Do các cột trong Diabetes BRFSS thiếu tính tương quan không gian tự nhiên, mô hình Conv1D chỉ đóng vai trò thí nghiệm sư phạm. Hướng phát triển tối ưu hơn trên bảng là ứng dụng các kiến trúc Tabular Transformer (như FT-Transformer hoặc TabNet) hoặc mô hình hóa dưới dạng Đồ thị Tri thức (Knowledge Graph).

3. Chưa áp dụng kỹ thuật cân bằng dữ liệu nâng cao: Để đảm bảo tính công bằng khi so sánh kiến trúc, các kỹ thuật như lấy mẫu lại (SMOTE), trọng số hàm mất mát (Weighted CrossEntropyLoss) hay Focal Loss chưa được tích hợp, dẫn tới việc chỉ số Positive-class Recall trên lớp bệnh nhân tiểu đường (17.6%) vẫn còn ở mức khiêm tốn.

11. KẾT LUẬN

Assignment 05 đã hoàn thành xuất sắc toàn bộ các yêu cầu nghiên cứu và thực nghiệm về Mạng nơ-ron tích chập (CNN). Thông qua việc cài đặt và đánh giá 12 mô hình PyTorch trên 3 tập dữ liệu đa dạng, báo cáo rút ra các kết luận cốt lõi sau:

1. Làm chủ sâu sắc lý thuyết và thực hành: Toàn bộ 14 khái niệm nền tảng của CNN, chu trình tiền xử lý chống rò rỉ dữ liệu, vòng lặp huấn luyện lan truyền ngược với Adam và hệ thống đánh giá đa chiều đã được hệ thống hóa chặt chẽ, minh họa bằng mã nguồn thực tế.

2. VGG-style dẫn đầu trên miền ảnh: Với triết lý xếp chồng các bộ lọc nhỏ 3x3 kết hợp

Batch Normalization và Dropout, VGG đạt hiệu năng cao nhất trên cả MNIST (Accuracy 98.68%, F1 0.9866) và Fashion-MNIST (Accuracy 89.98%, F1 0.9003).
3. ResNet-style tối ưu hóa tham số vượt bậc: Cơ chế Residual Learning cùng Global Average Pooling giúp ResNet đạt độ chính xác tương đương VGG trên MNIST nhưng tiết kiệm tới 64.4% số lượng tham số khả huấn luyện, đồng thời đạt Macro-F1 cao nhất (0.5913) trên dữ liệu bảng Diabetes.
4. Hiểu rõ bản chất dữ liệu và số đo thực nghiệm: Thực nghiệm minh chứng rõ rệt sự phân hóa giữa dữ liệu ảnh (phù hợp tối đa với tích chập 2D) và dữ liệu bảng (nơi các giả định quy nạp không gian bị suy giảm). Đặc biệt, phân tích chi tiết ma trận nhầm lẫn của ResNet1D chỉ ra rằng dù đạt Macro Recall 0.5735 và Accuracy 86.03%, độ nhạy riêng biệt của lớp bệnh nhân chỉ đạt 17.6%, chứng minh tầm quan trọng của việc không phụ thuộc vào số đo Accuracy trong các bài toán thực tế có độ lệch lớp cao.

Báo cáo này cung cấp tài liệu kỹ thuật hoàn chỉnh, chuẩn mực học thuật cao, phục vụ hiệu quả cho công tác nghiên cứu và triển khai các hệ thống thông minh ứng dụng học sâu trong thực tế.

12. TÀI LIỆU THAM KHẢO

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, 'Gradient-based learning applied to document recognition,' Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] K. Simonyan and A. Zisserman, 'Very deep convolutional networks for large-scale image recognition,' in International Conference on Learning Representations (ICLR), 2015. arXiv:1409.1556.

[3] K. He, X. Zhang, S. Ren, and J. Sun, 'Deep residual learning for image recognition,' in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[4] S. Ioffe and C. Szegedy, 'Batch normalization: Accelerating deep network training by reducing internal covariate shift,' in International Conference on Machine Learning (ICML), 2015, pp. 448–456.

[5] D. P. Kingma and J. Ba, 'Adam: A method for stochastic optimization,' in International Conference on Learning Representations (ICLR), 2015. arXiv:1412.6980.

[6] H. Xiao, K. Rasul, and R. Vollgraf, 'Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,' 2017. arXiv:1708.07747.

[7] Centers for Disease Control and Prevention (CDC), 'Behavioral Risk Factor Surveillance System (BRFSS) 2015 Survey Data and Documentation,' U.S. Department of Health and Human Services, 2015.

[8] A. Paszke et al., 'PyTorch: An imperative style, high-performance deep learning library,' in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 8024–8035.